第三章 SQL 注入漏洞与防护——实验分析报告

任务 1：数据库与查询条件基础说明

1. 页面输入与数据库的关系

网站的登录、搜索、查询等功能，**必须从数据库读取 / 写入数据**才能实现，用户在页面输入的内容会直接作为数据库查询的条件。

2. 最小业务链路

输入条件 → 数据库查询 → 返回结果以用户搜索为例：

1. 用户在搜索框输入 **ID=1**
2. 后端拼接 SQL：`SELECT * FROM users WHERE id=1`
3. 数据库执行并返回 ID=1 的用户信息

3. 输入进入查询条件的方式

后端代码**直接将用户输入拼接到 SQL 字符串**中，未做任何隔离处理，输入内容会成为 SQL 语句的一部分。

4. SQL 注入的产生原因

输入未被区分 **“数据”** 和 **“指令”**，攻击者输入 1’ OR 1=1#时，会改变原 SQL 结构，导致查询逻辑被篡改。

任务 2：DVWA SQL Injection Low 现象记录

主页 win10 serve win10 client

Welcome :: Damn Vulnerable Web

不安全 192.168.25.183/dvwa/index.php

DVWA

- Home
- Instructions
- Setup / Reset DB

- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography
- API

- DVWA Security
- PHP Info
- About

- Logout

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to **practice some of the most common web vulnerabilities**, with **various levels of difficultly**, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any Internet facing servers**, as they will be compromised. It is recommend using a virtual machine (such as VirtualBox or VMware), which is set to NAT networking mode. Inside a guest machine, you can download and install XAMPP for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

More Training Resources

DVWA aims to cover the most commonly seen vulnerabilities found in today's web applications. However there are plenty of other issues with web applications. Should you wish to explore any additional attack vectors, or want more difficult challenges, you may wish to look into the following other projects:

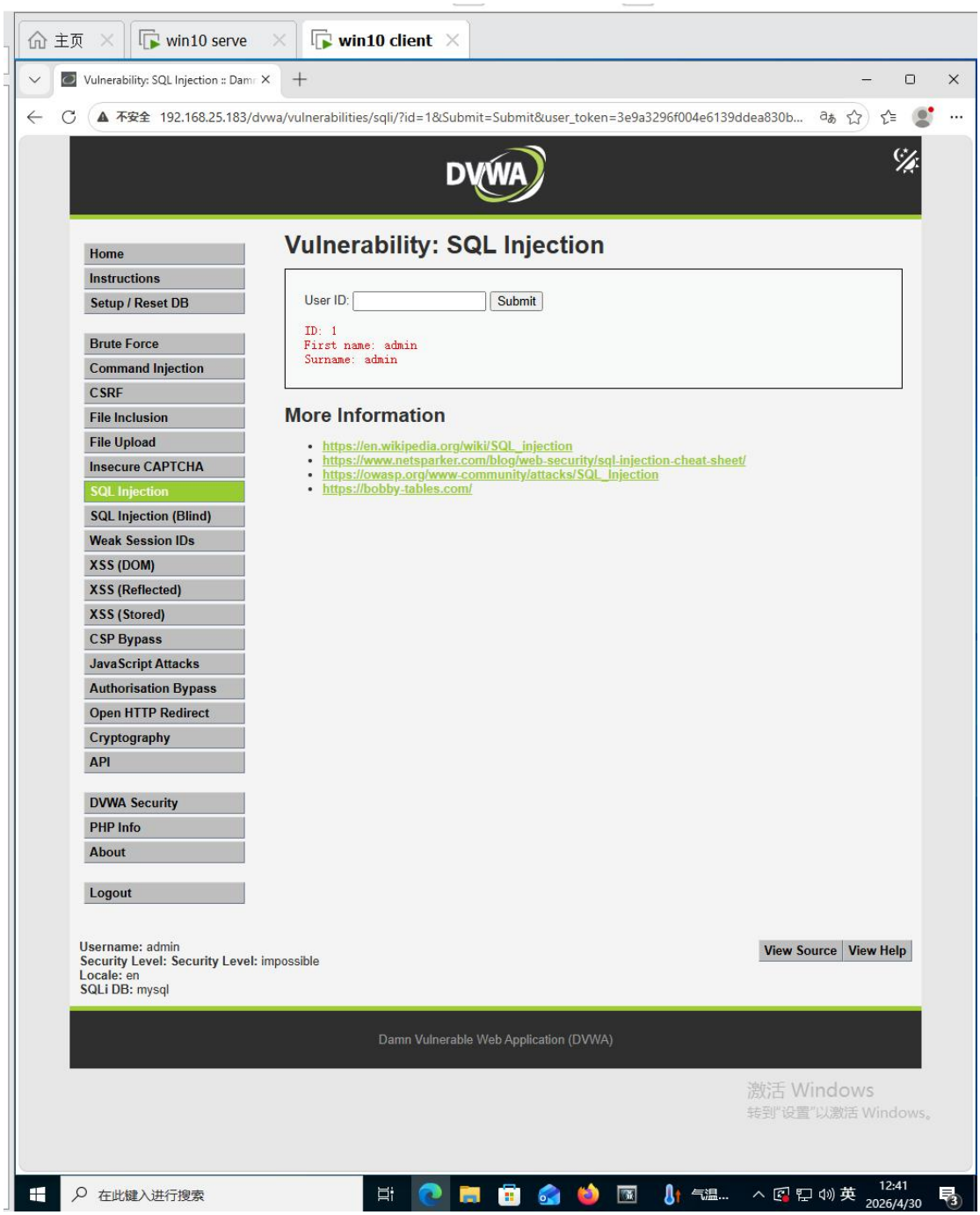
- Mutillidae
- OWASP Vulnerable Web Applications Directory

You have logged in as 'admin'

Username: admin
Security Level: **Security Level:** impossible
Locale: en
SQLi DB: mysql

激活 Windows
转到"设置"以激活 Windows。

在此键入进行搜索 17°C 英 12:39 2026/4/30



3. **结果差异与判断**

- 正常输入：仅返回单条数据，符合业务逻辑。
- 异常输入：返回全部数据，**超出正常查询范围**，证明输入直接修改了 SQL 结构，存在注入漏洞。

4. **为什么这种差异不像“正常查询变化”**

正常的 ID 查询只会根据输入的数字返回对应单条数据，输入不同 ID 只会切换单条结果；但异常输入后，要么返回全量数据、要么直接报错，完全脱离了“按 ID 查单条用户”的正常业务逻辑，说明后台未对输入做过滤，直接拼接了用户输入到 SQL 语句中，导致 SQL 语法被篡改，触发了注入漏洞，因此这种结果差异不属于正常查询的变化。

5.

任务 3 机制分析与差异比较

1. 完整注入链路

输入 → 字符串拼接 → 数据库解析 → 恶意执行 → 回显异常结果

2. 根因判断

SQL 注入根因是 B. **输入进入了不该进入的查询结构**理由：危险字符本身无害，只有被拼入 SQL 结构、改变查询逻辑时，才会形成漏洞。

3. Low 与 Medium 差异

- **Low**：无任何过滤，直接拼接输入，漏洞完全暴露。
- **Medium**：对'转义、限制输入类型，**仅做表层过滤**，未解决结构污染问题。

6. 普通注入与盲注差异

最大差异是 B. **证据形式不同**

- 普通注入：**有回显**，可直接看到数据。
- 盲注：**无回显**，通过页面真假、响应时间判断结果。
- 相同点：根因都是**查询结构被污染**。

任务 4 SQL 注入防护清单与解释

防护措施	解决问题	控制层级	不能替代
参数化查询	分离数据与 SQL 结构，从源头阻止拼接	**根因控制**	不能替代权限控制
输入校验	拦截非法字符，缩小攻击面	**暴露面控制**	不能阻止结构污染
错误处理	关闭数据库报错，避免泄露结构	**暴露面控制**	不能修复漏洞本身
最小权限	限制数据库账号权限，降低危害	**后果范围控制**	不能阻止注入发生

收口项 1：SQL 注入因果链/防护关系图

用户输入点 ↓（未过滤） 查询构造（直接拼接→结构污染） ↓ 数据库解析/执行（执行恶意 SQL） ↓ 结果差异（回显/盲注行为） ↓ 根因：结构被污染 防护控制点 1：参数化查询（切断拼接） 防护控制点 2：最小权限（缩小危害）

收口项 2：学生自评表

1. 最清楚：SQL 注入根因是**查询结构被污染**，不是危险字符。
2. 最易混淆：Low 与 Medium 的防护逻辑、盲注与普通注入的区别。
3. 需复核：任务 3 机制分析与判断题理由。
4. 优先改进：补充更多实验截图与现象对比。

收口项 3 AI 输出审核与人工复核记录（必做）

1. AI 辅助：作业框架、防护措施整理、术语规范。
2. 人工保留：根因判断、实验现象、差异对比等核心结论。
3. 修正内容：删除 AI 错误的“字符是根因”表述，统一为**结构污染**。
4. 验证方式：对照 DVWA 实验现象，确保结论与证据一致。